

教學實作：擴充 SG90 伺服馬達功能

步驟 1：修改硬體定義提示詞 (Devices Definition)

讓 AI 認識伺服馬達及其物理規則。

SG90 servo: pin 12

- servo angle control
- range: 0 - 180
- 0 = fully closed
- 180 = fully open

步驟 2：定義工具 JSON 規範 (Tools Definition)

在 `tools_definition` 中加入 `/servo` 指令的格式規範。

Servo motor control:

```
{
  "type": "tool_call",
  "method": "/servo",
  "params": {
    "pin": "<Device pin number. If the user does not specify a pin, ask first.>",
    "angle": "<Desired absolute angle from 0 to 180>"
  }
}
```

Success response:

```
{
  "status": "success",
  "method": "/servo",
  "pin": 12,
  "angle": 90
}
```

Error response:

```
{
  "status": "error",
  "method": "/servo",
  "reason": "undefined_servo_pin",
  "pin": 3
}
```

步驟 3：底層 C++ 程式碼實作

在專案中新增 tool_servo 驅動函數。

```
#include <AmebaServo.h>
AmebaServo servol2;

// Control a servo motor's position by specifying a target angle.
// This function supports precise physical movement for actuators
// like the SG90 servo connected to GPIO pins.
String tool_servo(AmebaServo &servo, int pin, int angle) {
    if (!servo.attached())
        servo.attach(pin);
    angle = constrain(angle, 0, 180);
    servo.write(angle);

    return
        "{\"status\":\"success\",",
        "\"method\":\"servo\",",
        "\"pin\":\"" + String(pin) + "\",",
        "\"angle\":\"" + String(angle) + "\"}";
}

void setup() {
    // ... 原有初始化內容 ...
    servol2.attach(12);
}
```

步驟 4：更新工具分發器邏輯 (useTools)

將 /servo 方法註冊進系統的分發流程。

```
void executeTool(String command, JsonObject params, bool reCheck = true) {

    if (command == "/digitalwrite" || command == "/analogwrite") {
        // ... 原有內容不變 ...
    }
    else if (command == "/servo") {
        int pin    = params["pin"].as<int>();
        int angle = params["angle"].as<int>();

        String response = "";
        if (pin == 12)
            response = tool_servo(servo12, pin, angle);
        else
            response = "{\"status\":\"error\","
                "\"reason\":\"undefined_servo_pin\","
                "\"pin\":\"" + String(pin) + "\"}";

        historicalMessages += buildGeminiMessage("user", command + timestamps);
        historicalMessages += buildGeminiMessage("model", response + timestamps);

        executeToolHistory += workId + " " + command + " [ " + String(pin) + " | " + String(angle) + " ]\n";

        evaluateWorkflowContinuation(workId, reCheck);
    }
}
```

```
else if (command == "/digitalread" || command == "/analogread") {  
    // ... 原有內容不變 ...  
}  
// ... 其餘 tools 不變 ...  
}
```